

Lecture 23 & 24 - Module 6. Ensemble learning

COMP 551 Applied machine learning

Yue Li

Assistant Professor
School of Computer Science
McGill University

April 3 & 8, 2025

Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

Least square additive boosting

AdaBoost

Gradient boosting

Summary

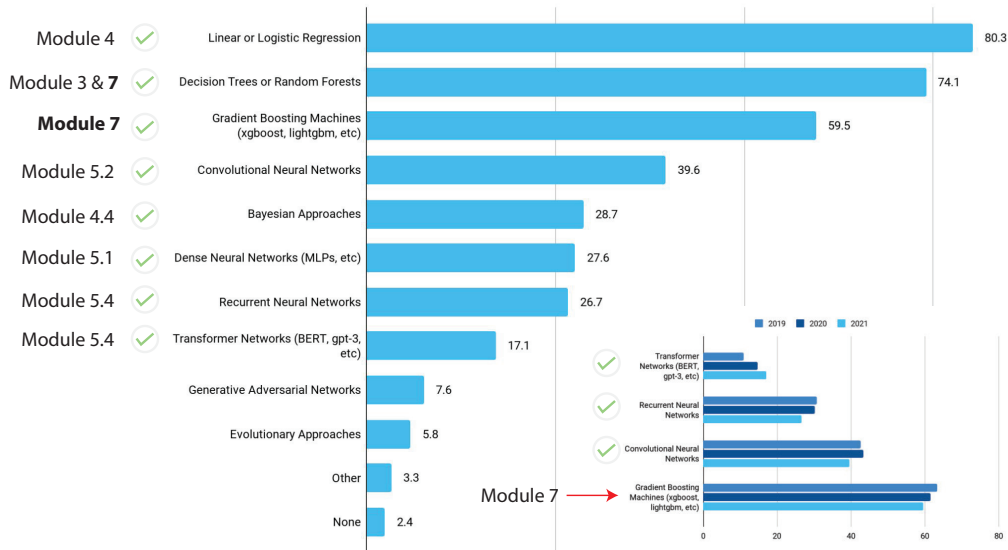
Appendix (optional)

Deriving AdaBoost loss function

Acknowledgement

- The slides are primarily prepared based on Chapter 18.2 from the Murphy 2022 complemented by Bishop06 Chapter 14.3.
- The bias-variance trade-off slides are adapted from COMP 551 Winter 2022 from Prof. Reihaneh Rabbany

Most commonly used ML algorithms in Kaggle 2021 survey

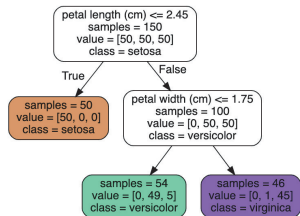


Learning objectives

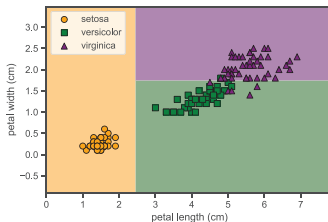
- Advantages of ensemble learning
- Bootstrap aggregation (Bagging)
- Random Forest
- Boosting (optional)
- AdaBoost (optional)
- GradientBoosting (optional)

Decision tree (Module 3) is sensitive to the change of training data

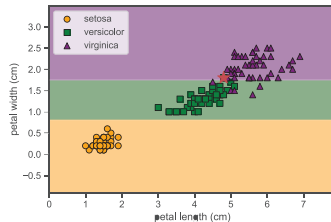
DT (tree depth = 2) for Iris flower



Decision surface of the DT



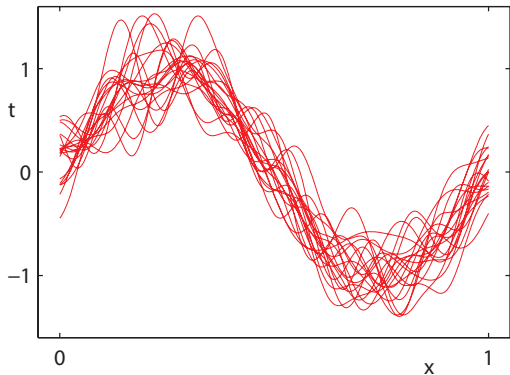
Fit to the same data but omit a single data point (shown by star)



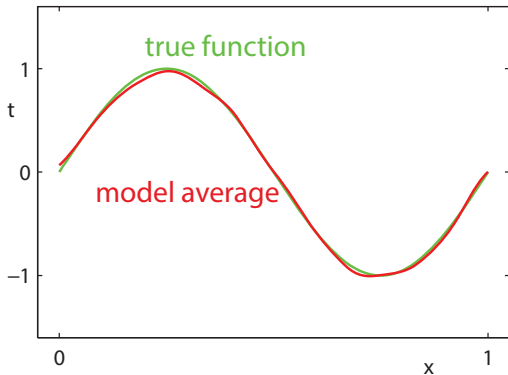
Decision trees are unstable: small changes to the input data can have large effects on the structure of the tree, due to the greedy and hierarchical nature of the tree-growing process.

Recall in Module 4.5: model averaging reduces variance

Each curve is predicted by a distinct model $\hat{f}_m(\mathbf{x})$



Averaged prediction by all models: $\hat{y} = \sum_m \hat{f}_m(\mathbf{x})$



$$\mathbb{E}_{\mathcal{D}} \left[(\hat{f}(\mathbf{x}; \mathcal{D}) - y)^2 \right] = \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\hat{f}_{\mathcal{D}}(\mathbf{x}) - \mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] \right)^2 \right]}_{\text{Variance}} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\left(\mathbb{E}_{\mathcal{D}}[\hat{f}_{\mathcal{D}}(\mathbf{x})] - f(\mathbf{x}) \right)^2 \right]}_{\text{Bias}} + \underbrace{\mathbb{E}_{\mathcal{D}} \left[\epsilon^2 \right]}_{\text{noise}}$$

Ensemble learning reduces variance by M times for independent models

Assuming $f_m(x) = z_m$ for $m = 1, \dots, B$. Variance of the two random variables z_1 and z_2 is:

$$\text{Var}[z_1 + z_2] = \text{Var}[z_1] + \text{Var}[z_2] + 2 \underbrace{\text{Cov}[z_1, z_2]}_{0 \text{ if } z_1 \perp z_2}$$

Variance of the sum of B independent variables (the last equality assumes equal variance σ^2):

$$\text{Var}\left[\sum_{m=1}^M z_m\right] = \sum_{m=1}^M \text{Var}[z_m] \triangleq M\sigma^2$$

Variance of the average over M independent variables is M times smaller than their individual variances:

$$\text{Var}\left[\frac{1}{M} \sum_{m=1}^M z_m\right] = \frac{1}{M^2} \text{Var}\left[\sum_{m=1}^M z_m\right] = \frac{1}{M^2} M\sigma^2 = \frac{1}{M} \sigma^2$$

Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

- Least square additive boosting

- AdaBoost

- Gradient boosting

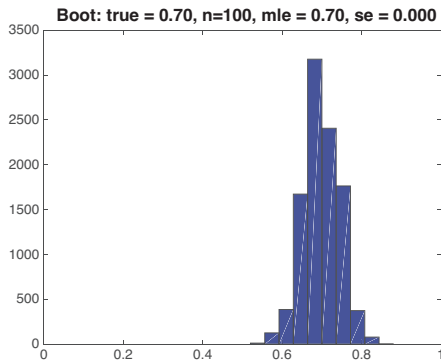
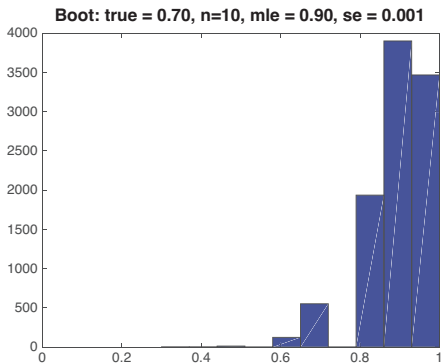
Summary

Appendix (optional)

- Deriving AdaBoost loss function

Bootstrap sampling

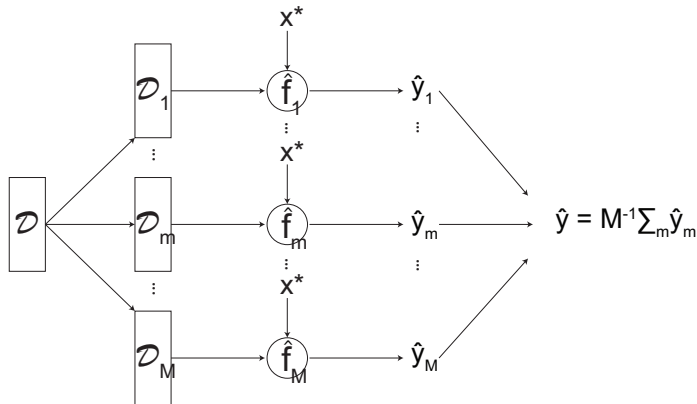
1. Suppose we have a single dataset $\mathcal{D}$ of size N (e.g., N coin tosses).
2. Sample *with replacement* to create M sets of datasets $\mathcal{D}^{(m)}$ each of size N .
3. For each dataset $\mathcal{D}^{(m)}$, train a model to obtain model parameter $\theta^{(m)}$ (e.g., the MLE of the Bernoulli rate from N coin tosses in $\mathcal{D}^{(m)}$ is $\theta^{(m)} = \frac{1}{N} \sum_{n=1}^N y_n^{(m)}$).
4. Average over the model parameters $\theta^{(m)}$ to approximate $\mathbb{E}_{\mathcal{D}}[\theta]$.
5. Histograms based on $B = 10K$ bootstraps each with $N=10$ and 100 samples.



Bagging: Bootstrap aggregation (L. Breiman. Bagging predictors. In: *Machine Learning* 24 (1996))

Bagging training procedure is simple:

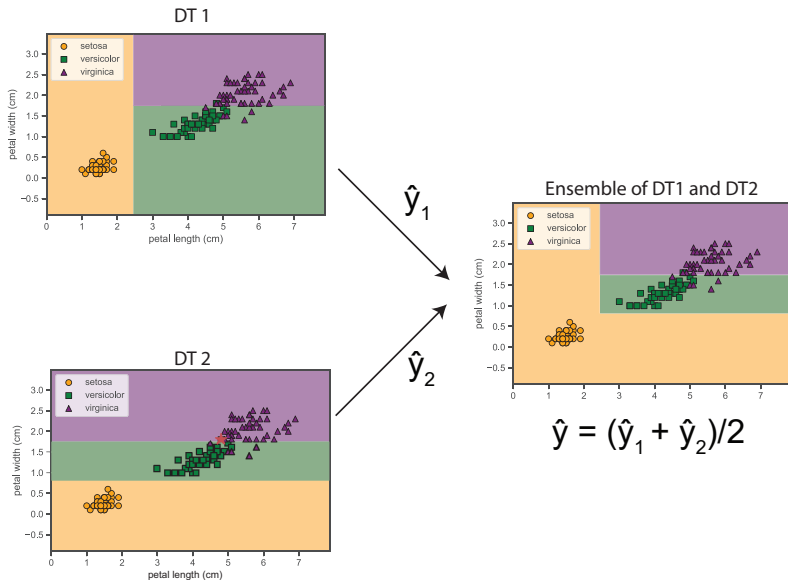
1. Suppose we have a labelled dataset $\mathcal{D} = \{\mathbf{x}_n, y_n\}_{n=1}^N$.
2. Sample *with replacement* from $\mathcal{D}$ to get M datasets $\mathcal{D}^{(m)} = \{\mathbf{x}_n^{(m)}, y_n^{(m)}\}_{n=1}^N$.
3. Train M **base models** (or **weak learners**) on the M datasets.
4. During testing, average the model predictions.



Properties of ensemble models trained by the bagging algorithm

- Each model is trained on average, 63% of the *unique* examples:
 - ▶ The chance that a single data point will *not* be selected from the set of size N in any of the N draws is $(1 - 1/N)^N$
 - ▶ In the limit for large N , $\lim_{N \rightarrow \infty} (1 - 1/N)^N \rightarrow \exp(-1) \approx 0.37$
- The 37% of the training instances that are not used by a given base model are called **out-of-bag** (OOB) instances, which can be used as validation set for the base model (e.g., choosing tree depth).
- Bagging prevents the ensemble from relying too much on any individual training example, which improves robustness and generalization
- However, bagging does not always improve performance. The base model needs to be an **unstable estimator** so that omitting some data changes the model fit.
- Also, the more independent the weak learners the better
- **Classification and regression trees** (CARTs) satisfy both conditions above but not other methods like KNN, linear regression or neural networks.

Ensemble of two decision trees fit to the Iris flower dataset



Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

- Least square additive boosting

- AdaBoost

- Gradient boosting

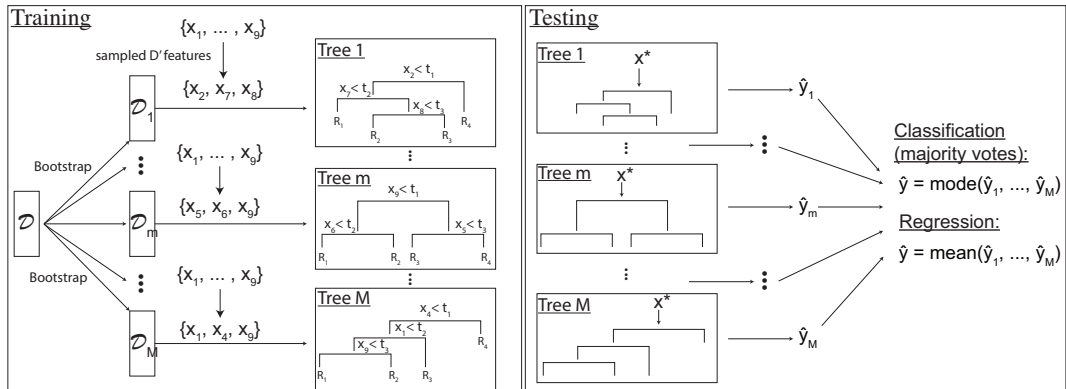
Summary

Appendix (optional)

- Deriving AdaBoost loss function

Random Forest (RF) [L. Brieman *Machine Learning* 45.1 (2001), pp. 5-32]

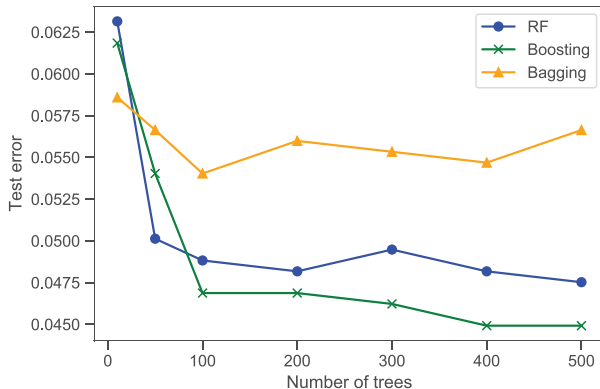
- Bagging learns diverse base models (i.e., weak classifiers) on subsets of bootstrap-sampled training examples. RF seeks to further *decorrelate* the weak classifiers (i.e., decision tree).
- An RF learns K decision or regression trees, where each tree is trained based on
 1. a bootstrap sample of N training data points and
 2. a randomly chosen subset of $\sqrt{D}$ (by default) input features at each tree node
- Out of bag (OOB) samples for each tree are used as validation to determine tree depth.



Spam detection using hand-crafted tabular data of 57 features

4601 email messages labelled with spam ($y = 1$) or non-spam ($y = 0$). The data was made available by George Forman from Hewlett-Packard. The 57 input features are:

- 48 keyword frequencies (e.g., business, address, internet, George (user name)),
- 6 special characters (; . [\$ #),
- 3 features corresponding to average, max, total length of capital letters.



RF performs much better than Bagging but worse than Boosting (discussed shortly).

Computing feature importance from RF

- Trees are popular because they are interpretable.
- Ensemble of trees such as RF lose that property
- To extract feature importance from a trained RF, we can follow these steps:
 1. Compute feature importance score for each feature d from a single decision tree classifier m by summing over all non-leaf nodes where feature d is used weighted by the reduction of cost (e.g., gini-index) (see Module 3 DT):

$$r_d(T_m) = \sum_{j=1}^J \Delta g_j \mathbb{I}[v_j = d]$$

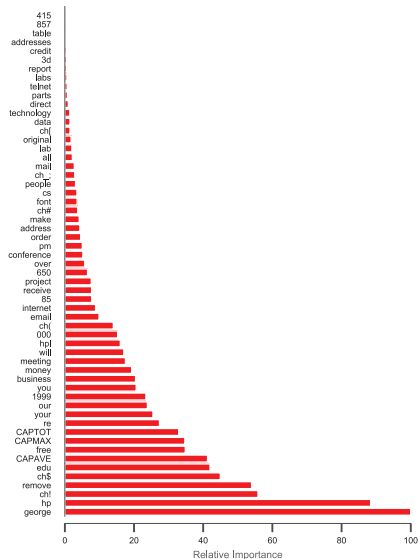
2. Average over all trees in the RF:

$$r_d = \frac{1}{M} \sum_{m=1}^M r_d(T_m)$$

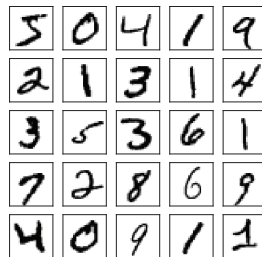
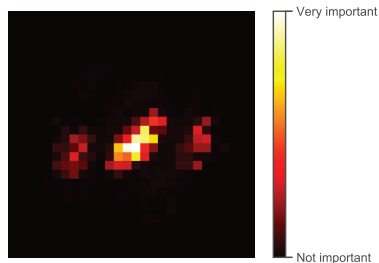
3. Normalize the score: $\hat{r}_d = 100 \times r_d / \max(\mathbf{r})$

Feature importance scores for spam detection and MNIST classification

Feature importance of spam detection



Feature importance of MNIST classification



Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

- Least square additive boosting

- AdaBoost

- Gradient boosting

Summary

Appendix (optional)

- Deriving AdaBoost loss function

Boosting is a sequential fitting algorithm

Let each model F_m be a weak classifier.

1. Fit function F_1 on the original training data
2. For F_m model, weight the data samples by the errors made by f_{m-1} so the misclassified examples get more weight
3. Fit F_m and its weight β_m to the weighted data set
4. Repeat 2 and 3 until we have the desired M models $F_1, \dots, F_M$.
5. The final ensemble model prediction is

$$f(\mathbf{x}, \Theta) = \sum_m \beta_m F_m(\mathbf{x}; \theta_m)$$

Boosting versus RF/bagging:

- When $\beta_m = 1/M$ and each model F_m is fit separately to the subsampled of the data and/or features, we recover bagging or RF.
- While bagging and RF reduce the variance by fitting independent trees, boosting reduces the bias by *sequentially* fitting classifiers that depend on each other.
- While boosting is slower than bagging and RF because of its sequential fitting algorithm, in practice it often produces better performance as we saw in the earlier spam detection application.

Forward stagewise additive modelling

Goal: minimize the prediction loss on N training data points w.r.t. the function:

$$\mathcal{L}(f) = \sum_{n=1}^N \ell(y^{(n)}, f(\mathbf{x}^{(n)}))$$

Strategy: sequentially minimize the loss at each iteration m :

1. Obtain optimal parameters for the base model function $F(\mathbf{x}, \theta)$ (**weak learner**) and its weight β :

$$\beta_m, \theta_m \leftarrow \arg \min_{\beta, \theta} \sum_{n=1}^N \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}) + \beta F(\mathbf{x}^{(n)}; \theta))$$

2. Set the ensemble function (**strong learner**) at the iteration m to be:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \beta_m F(\mathbf{x}; \theta_m)$$

Note: the detailed boosting algorithm depends on the loss function ℓ . The most intuitive loss function in this context is the *quadratic loss* and the fitting algorithm is called **least squares boosting** (L2Boosting) (next).

Quadratic loss and least squares boosting (L2Boosting)

Squared error loss:

$$\begin{aligned}\ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}) + \beta F(\mathbf{x}^{(n)}; \theta)) &\equiv (y^{(n)} - f_{m-1}(\mathbf{x}^{(n)}) - \beta F(\mathbf{x}^{(n)}; \theta))^2 \\ &\equiv (r_m^{(n)} - \beta F(\mathbf{x}^{(n)}; \theta))^2\end{aligned}$$

- $r_m^{(n)} = y^{(n)} - f_{m-1}(\mathbf{x}^{(n)})$ is the current residual error on the n 'th training example.
- Therefore, $\beta F(\mathbf{x}^{(n)}; \theta)$ further reduces the residual error made by $f_{m-1}(\mathbf{x}^{(n)})$.
- Overall fitting algorithm of least squares boosting then becomes:

$$\begin{aligned}\beta_m, \theta_m &\leftarrow \arg \min_{\beta, \theta} \sum_{i=1}^N (r_{im} - \beta F(\mathbf{x}^{(i)}; \theta))^2 \\ f_m(\mathbf{x}^{(n)}) &= f_{m-1}(\mathbf{x}^{(n)}) + \beta_m F(\mathbf{x}^{(n)}; \theta_m)\end{aligned}$$

- Note: the function $F(\mathbf{x}^{(n)}; \theta)$ does not need to be differentiable. For example, we can use regression tree as the base model.

Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

Least square additive boosting

AdaBoost

Gradient boosting

Summary

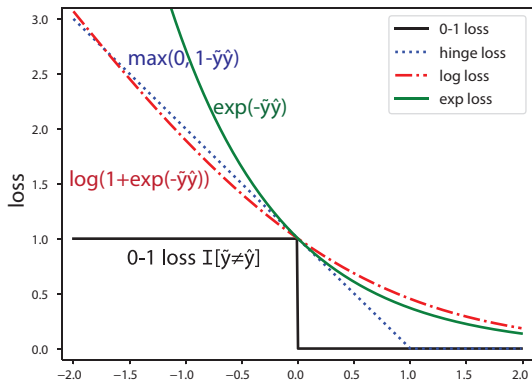
Appendix (optional)

Deriving AdaBoost loss function

Exponential loss for binary classification $\tilde{y} \in \{-1, +1\}$

Here we consider *exponential loss* because of the mathematical convenience in minimizing this loss in a boosting model called AdaBoost (described next):

$$\ell(\tilde{y}, F(\mathbf{x})) = \exp(-\tilde{y}F(\mathbf{x}))$$



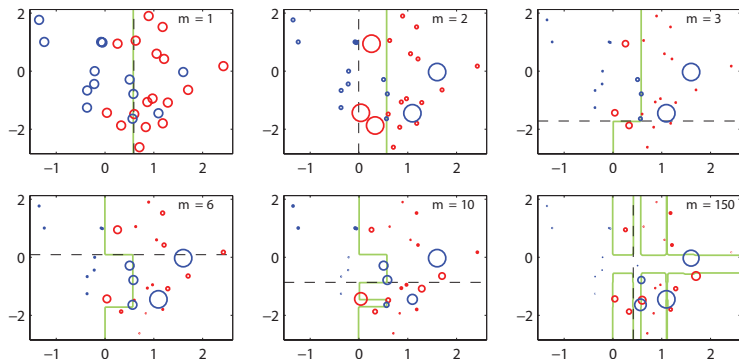
AdaBoost learning algorithm

Algorithm 1 Adaboost.fit($\mathbf{X}$, $\mathbf{y}$, max_iter= M)

- 1: $\omega_1^{(n)} = 1/N$
 - 2: **for** $m = 1 \dots \text{max_iter}$ **do**
 - 3: $\hat{F}_m = \arg \min_F \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F(\mathbf{x}^{(n)})]$
 - 4: $err_m = \frac{\sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})]}{\sum_n \omega_m^{(n)}}$
 - 5: $\alpha_m = \log \frac{1 - err_m}{err_m}$
 - 6: $\omega_{m+1}^{(n)} \leftarrow \omega_m^{(n)} \exp(\alpha_m \mathbb{I}[\tilde{y}^{(n)} \neq \hat{F}_m(\mathbf{x})])$
 - 7: **end for**
 - 8: Return $f(\mathbf{x}) = \text{sgn} \left[\sum_{m=1}^M \alpha_m \hat{F}_m(\mathbf{x}) \right]$
-

Derivation is provided in the Appendix [1](#)

AdaBoost modeling fitting for 150 iterations (Bishop06 Fig. 14.2)



- Each base learner $F_m(\mathbf{x})$ consists of simply thresholding applied to the x or y -axis as indicated by their black decision boundaries.
- m indicates the number of base learners trained so far.
- Green decision boundary is the ensemble function $f_m(\mathbf{x})$.
- Size of the data point is proportional to their weight $\omega_m^{(n)}$.
- For example, the points that are misclassified by the $m=1$ base learner are given greater weight when training the $m=2$ learner and so on.

Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

Least square additive boosting

AdaBoost

Gradient boosting

Summary

Appendix (optional)

Deriving AdaBoost loss function

Gradient boosting

- As we can see in L2Boost and AdaBoost, changing the loss function involves deriving a new boosting algorithm, which can be technically challenging.
- Gradient boosting (GB) provides a generic framework “sandbox” that works with different differentiable loss functions
- GB performs gradient descent in **w.r.t. the function outputs**:

$$\nabla f_{m-1}(\mathbf{x}^{(n)}) = \frac{\partial \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})} \equiv -r_m^{(n)}$$

- We then train a base function to *predict* the gradient of the function by minimizing the squared differences between the base function predictions and the gradients:

$$F_m = \arg \min_{F_m} \sum_{n=1}^N (r_m^{(n)} - F_m(\mathbf{x}^{(n)}))^2$$

- Finally, we update the function:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) - \nu \nabla f_{m-1}(\mathbf{x}^{(n)}) \approx f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$$

where $\nu \in [0, 1]$ is the learning rate.

Gradient boosting algorithm

Algorithm 2 Gradient boosting.fit(**X**, **y**, max_iter= M , $\nu = 1$)

- 1: Initialize $f_0(\mathbf{x}) = \arg \min_F \sum_{n=1}^N L(y, F(\mathbf{x}^{(n)}))$
 - 2: **for** $m = 1 \dots \text{max_iter}$ **do**
 - 3: Compute the gradient residual $r_m^{(n)} = -\frac{\partial \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})}$
 - 4: Train a weak learner $F_m = \arg \min_{F_m} \sum_{n=1}^N (r_m^{(n)} - F_m(\mathbf{x}^{(n)}))^2$
 - 5: Update ensemble learner $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$
 - 6: **end for**
 - 7: Return $f_M(\mathbf{x})$
-

Some common loss function and their negative gradients:

	Loss	$-\partial \ell(y, f(x)) / \partial f(x)$		Loss	$-\partial \ell(y, f(x)) / \partial f(x)$
Squared error	$\frac{1}{2}(y - f(x))^2$	$y - f(x)$	Binary Log	$\log(1 + e^{-\tilde{y}f(x)})$	$y - f(x)$
Absolute error	$ y_i - f(x) $	$\text{sgn}(y - f(x))$	Multiclass Log	$y_c \log f_c(x)$	$y_c - f_c(x)$
Exponential loss	$\exp(-\tilde{y}f(x))$	$-\tilde{y} \exp(-\tilde{y}f(x))$			

Gradient boosting tree

- Why trees? Trees are flexible, sensitive to subsamples of data and/or features.
- Recall that a regression tree function is defined as:

$$F_m(\mathbf{x}) = \sum_{j=1}^{J_m} w_{j,m} \mathbb{I}(\mathbf{x} \in R_{j,m}), \quad \text{where } j \text{ is the region in the leaf node}$$

Algorithm 3 Gradient boosting **tree**($\mathbf{X}, \mathbf{y}, \text{max_iter}=M, \nu = 1$) *colored text are tree-specific*

- 1: Initialize $f_0(\mathbf{x}) = \arg \min_F \sum_{n=1}^N L(y, F(\mathbf{x}^{(n)}))$
- 2: **for** $m = 1 \dots \text{max_iter}$ **do**
- 3: Compute the gradient residual $r_m^{(n)} = -\frac{\partial \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})} \triangleq y^{(n)} - f_{m-1}(\mathbf{x}^{(n)})$ (Δ squared loss)
- 4: **Learn structure of the tree model** m : $F_m = \arg \min_{F_m} \sum_{n=1}^N (r_m^{(n)} - F_m(\mathbf{x}^{(n)}))^2$
- 5: **Re-adjust leaf weights**: $\hat{w}_{j,m} \leftarrow \arg \min_{w_{j,m}} \sum_{n \in \mathcal{R}_{j,m}} \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}) + w_{j,m} \mathbb{I}(\mathbf{x}^{(n)} \in R_{j,m}))$
- 6: Update ensemble learner $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$
- 7: **end for**
- 8: Return $f_M(\mathbf{x})$

Extreme gradient boosting (XGBoost)

- XGBoost adapts GBT by first adding a regularizer to the loss function:

$$\mathcal{L}(f) = \sum_{n=1}^N \ell(y^{(n)}, f(\mathbf{x}^{(n)})) + \Omega(f), \quad \text{where } \Omega(f) = \gamma J + \frac{1}{2} \lambda \sum_{j=1}^J w_j^2$$

where J is the number of leaves and w_j is the regression weight at leaf j .

- Unlike gradient boosting that works as gradient descent in function space, XGBoost approximates the loss by second order Taylor expansion:

$$\begin{aligned} \mathcal{L}_m(F_m) &= \sum_{n=1}^N \ell(y^{(n)}, f_{m-1}(\mathbf{x}) + F_m(\mathbf{x}^{(n)})) + \Omega(F_m) \\ &\approx \sum_{n=1}^N \left[\underbrace{\ell(y^{(n)}, f_{m-1}(\mathbf{x}))}_{\text{constant w.r.t. } F_m} + g_m^{(n)} F_m(\mathbf{x}^{(n)}) + \frac{1}{2} h_m^{(n)} F_m^2(\mathbf{x}^{(n)}) \right] + \Omega(F_m) \end{aligned}$$

where $g_m^{(n)}$ and $h_m^{(n)}$ are the first and second-order gradients (i.e., the Hessian):

$$g_m^{(n)} = \frac{\partial \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})}, \quad h_m^{(n)} = \frac{\partial^2 \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})^2}$$

XGBoost using regression tree

- Recall that a regression tree function is defined as: $F_m(\mathbf{x}^{(n)}) = \sum_{j=1}^{J_m} \sum_{n \in R_{j,m}} w_{j,m}$
- Substituting $F_m(\mathbf{x}^{(n)})$ and rewriting the loss function gives:

$$\begin{aligned}\mathcal{L}_m(F_m) &\approx \sum_{n=1}^N \left[g_m^{(n)} F_m(\mathbf{x}^{(n)}) + \frac{1}{2} h_m^{(n)} F_m^2(\mathbf{x}^{(n)}) \right] + \gamma J_m + \frac{1}{2} \lambda \sum_{j=1}^{J_m} w_{j,m}^2 \\ &= \sum_{j=1}^{J_m} \left[\left(\sum_{n \in R_{j,m}} g_{j,m}^{(n)} \right) w_{j,m} + \frac{1}{2} \left(\sum_{n \in R_{j,m}} h_{j,m}^{(n)} + \lambda \right) w_{j,m}^2 \right] + \gamma J_m \\ &= \sum_{j=1}^{J_m} \left[G_{j,m} w_{j,m} + \frac{1}{2} (H_{j,m} + \lambda) w_{j,m}^2 \right] + \gamma J_m\end{aligned}$$

- Solving $\frac{\partial \mathcal{L}_m(F_m)}{\partial w_{j,m}} = 0$ gives: $w_{j,m}^* = -\frac{G_{j,m}}{H_{j,m} + \lambda}$
- Plugging $w_{j,m}^*$ back into $\mathcal{L}_m(F_m)$ gives:

$$\mathcal{L}_m(F_m) = -\frac{1}{2} \sum_{j=1}^{J_m} \frac{G_{j,m}^2}{H_{j,m} + \lambda} + \gamma J_m$$

XGBoost algorithm

Algorithm 4 XGBoost ($\mathbf{X}, \mathbf{y}, \text{max_iter}=M, \nu = 1$)

1: Initialize $f_0(\mathbf{x}) = \arg \min_F \sum_{n=1}^N L(y, F(\mathbf{x}^{(n)}))$

2: **for** $m = 1 \dots \text{max_iter}$ **do**

3: Compute the first order and second order gradients:

$$g_{j,m}^{(n)} = \frac{\partial \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})}, \quad G_{j,m} = \sum_{n \in R_{j,m}} g_{j,m}^{(n)}, \quad h_{j,m}^{(n)} = \frac{\partial^2 \ell(y^{(n)}, f_{m-1}(\mathbf{x}^{(n)}))}{\partial f_{m-1}(\mathbf{x}^{(n)})^2}, \quad H_{j,m} = \sum_{n \in R_{j,m}} h_{j,m}^{(n)}$$

4: Learn structure of the base tree model F_m :

$$F_m = \arg \min_{F_m} - \frac{1}{2} \sum_{j=1}^{J_m} \frac{G_{j,m}^2}{H_{j,m} + \lambda} + \gamma J_m,$$

// Greedily split each node by the maximizing $gain = \frac{1}{2} \left[\frac{G_L^2}{H_L + \lambda} + \frac{G_R^2}{H_R + \lambda} - \frac{(G_L + G_R)^2}{H_L + H_R + \lambda} - \gamma \right]$,

where $G_L = \sum_{n \in R_L} g_m^{(n)}$, $G_R = \sum_{n \in R_R} g_m^{(n)}$, $H_L = \sum_{n \in R_L} h_m^{(n)}$, $H_R = \sum_{n \in R_R} h_m^{(n)}$.

5: Update ensemble learner $f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \nu F_m(\mathbf{x})$

6: **end for**

7: Return $f_M(\mathbf{x})$

Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

- Least square additive boosting

- AdaBoost

- Gradient boosting

Summary

Appendix (optional)

- Deriving AdaBoost loss function

Summary

Key Idea: Combine multiple weak/unstable models to improve performance via:

- **Variance reduction** (Bagging, Random Forest)
- **Bias reduction** (Boosting)

Bagging (Bootstrap Aggregating):

- Train M models on bootstrap samples
- Average (regression) or majority vote (classification)
- Effective for unstable learners (e.g., decision trees)
- Out-of-bag (OOB) estimation for validation

Random Forest:

- Bagging + random feature selection per node
- Improves tree de-correlation and performance
- Enables feature importance estimation

Boosting (Sequential Fitting):

- Focuses on previous model errors
- Reduces bias, often outperforms bagging

Takeaway: Ensemble methods stabilize ML models and perform well in practice.

Outline

Learning objectives

Ensemble learning

Bagging

Random Forest

Boosting (optional)

- Least square additive boosting

- AdaBoost

- Gradient boosting

Summary

Appendix (optional)

- Deriving AdaBoost loss function

Deriving AdaBoost loss function

Since $F(\mathbf{x}^{(n)}; \theta)$ does not need to be differentiable function, to not clutter the notation, we omit θ in $F(\mathbf{x}^{(n)}; \theta)$ and denote $F_m(\mathbf{x}^{(n)})$ as the base model in iteration m . Similar to L2Boosting, we start with the general loss function and plug in the exponential loss instead of squared loss:

$$\begin{aligned}\mathcal{L}_m^{(n)} &= \ell \left(\tilde{y}^{(n)}, f_{m-1}(\mathbf{x}^{(n)}) + \beta_m F_m(\mathbf{x}^{(n)}) \right) \\ &= \exp \left[-\tilde{y}^{(n)} \left(f_{m-1}(\mathbf{x}^{(n)}) + \beta_m F_m(\mathbf{x}^{(n)}) \right) \right] \\ &= \exp \left[-\tilde{y}^{(n)} f_{m-1}(\mathbf{x}^{(n)}) - \beta_m \tilde{y}^{(n)} F_m(\mathbf{x}^{(n)}) \right] \\ &= \exp \left[-\tilde{y}^{(n)} f_{m-1}(\mathbf{x}^{(n)}) \right] \exp \left[-\beta \tilde{y}^{(n)} F_m(\mathbf{x}^{(n)}) \right] \\ &\equiv \omega_m^{(n)} \exp \left[-\beta \tilde{y}^{(n)} F_m(\mathbf{x}^{(n)}) \right]\end{aligned}$$

where $\omega_m^{(n)}$ is the weight applied to data point n . The second last equality leverages the fact that the exponential of the sum is equal to the products of the exponentials.

We can further simplify the loss by considering two case scenarios:

$$\beta_m \tilde{y}^{(n)} F_m(\mathbf{x}^{(n)}) = \begin{cases} \beta_m & \text{if } \tilde{y}^{(n)} = F_m(\mathbf{x}^{(n)}) \\ -\beta_m & \text{if } \tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)}) \end{cases}$$

Denoting $\mathcal{L}_m \equiv \ell(\tilde{y}^{(n)}, f_{m-1}(\mathbf{x}^{(n)}) + \beta_m F_m(\mathbf{x}^{(n)}))$, we can rewrite the loss at iteration m as:

$$\begin{aligned} \mathcal{L}_m &= \sum_n \omega_m^{(n)} \exp \left[-\beta_m \tilde{y}^{(n)} F_m(\mathbf{x}^{(n)}) \right] = \sum_{\tilde{y}^{(n)} = F_m(\mathbf{x}^{(n)})} \omega_m^{(n)} \exp(-\beta_m) + \sum_{\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})} \omega_m^{(n)} \exp(\beta_m) \\ &= \exp(-\beta_m) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} = F_m(\mathbf{x}^{(n)})] + \exp(\beta_m) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})] \\ &= \exp(-\beta_m) \sum_n \omega_m^{(n)} (1 - \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})]) + \exp(\beta_m) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})] \\ &= \exp(-\beta_m) \sum_n \omega_m^{(n)} - \exp(-\beta_m) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})] + \exp(\beta_m) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})] \\ &= \exp(-\beta_m) \sum_n \omega_m^{(n)} + (\exp(\beta_m) - \exp(-\beta_m)) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})] \end{aligned}$$

Optimize β to weight the contribution of the base model

Lets denote $w \equiv \sum_n \omega_m^{(n)}$ and $r \equiv \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})]$

$$\mathcal{L}_m = \exp(-\beta_m) \sum_n \omega_m^{(n)} + (\exp(\beta_m) - \exp(-\beta_m)) \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})]$$

$$\equiv \exp(-\beta_m) w + (\exp(\beta_m) - \exp(-\beta_m)) r$$

$$\frac{\partial \mathcal{L}_m}{\partial \beta_m} = -w \exp(-\beta_m) + (\exp(\beta_m) + \exp(-\beta_m)) r = (r - w) \exp(-\beta_m) + r \exp(\beta_m) = 0$$

Solving for β :

$$0 = (r - w) \exp(-\beta_m) + r \exp(\beta_m)$$

$$0 = r - w + r \exp(2\beta_m)$$

$$\exp(2\beta_m) = \frac{w - r}{r}$$

$$\beta_m = \frac{1}{2} \log \frac{w - r}{r}$$

$$= \frac{1}{2} \log \frac{1 - r/w}{r/w} = \frac{1}{2} \log \frac{1 - err_m}{err_m}$$

where

$$err_m = \frac{\sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F_m(\mathbf{x}^{(n)})]}{\sum_n \omega_m^{(n)}}$$

Intuitively, the contribution β_m of model F_m is *inversely weighted* by its log error rate: $\frac{1}{2}(\log \frac{1}{err_m} - 1) \propto -\frac{1}{2} \log err_m$.

AdaBoost model updates

Minimizing $\mathcal{L}_m$ w.r.t. F_m is equivalent to minimizing its weighted misclassification rate:

$$\begin{aligned}\mathcal{L}_m &= \underbrace{\exp(-\beta) \sum_n \omega_m^{(n)}}_{\text{constant w.r.t. } F} + \underbrace{(\exp(\beta) - \exp(-\beta))}_{\text{constant w.r.t. } F} \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F(\mathbf{x}^{(n)})] \\ &\propto \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F(\mathbf{x}^{(n)})] \Rightarrow \hat{F}_m = \arg \min_F \sum_n \omega_m^{(n)} \mathbb{I}[\tilde{y}^{(n)} \neq F(\mathbf{x}^{(n)})]\end{aligned}$$

The overall update of the strong learner is then:

$$f_m(\mathbf{x}) = f_{m-1}(\mathbf{x}) + \hat{\beta}_m \hat{F}_m(\mathbf{x}) \quad \text{where} \quad \hat{\beta}_m \propto -\log \text{err}_m$$

The weight for each training example n in the next iteration then becomes:

$$\begin{aligned}\omega_{m+1}^{(n)} &= \exp(-\tilde{y}^{(n)} f_m(\mathbf{x}^{(n)})) &&= \omega_m^{(n)} \exp \left[-\hat{\beta}_m \left(1 - 2\mathbb{I}[\tilde{y}^{(n)} \neq \hat{F}_m(\mathbf{x})] \right) \right] \\ &= \exp(-\tilde{y}^{(n)} f_{m-1}(\mathbf{x}) - \hat{\beta}_m \tilde{y}^{(n)} \hat{F}_m(\mathbf{x})) &&= \omega_m^{(n)} \underbrace{\exp(-\hat{\beta}_m)}_{\text{constant}} \underbrace{\exp(2\hat{\beta}_m \mathbb{I}[\tilde{y}^{(n)} \neq \hat{F}_m(\mathbf{x})])}_{\alpha_m} \\ &= \omega_m^{(n)} \exp(-\hat{\beta}_m \tilde{y}^{(n)} \hat{F}_m(\mathbf{x})) &&\propto \omega_m^{(n)} \underbrace{\exp(\alpha_m \mathbb{I}[\tilde{y}^{(n)} \neq \hat{F}_m(\mathbf{x})])}_{\text{exponential error}} \\ &= \omega_m^{(n)} \exp \left[-\hat{\beta}_m \left(\mathbb{I}[\tilde{y}^{(n)} = \hat{F}_m(\mathbf{x})] - \mathbb{I}[\tilde{y}^{(n)} \neq \hat{F}_m(\mathbf{x})] \right) \right]\end{aligned}$$